

Great! Below is **Part 1** of the **style.css** for your **AppointmentBot AI**. This part includes:

- Animated Gradient Background
- Glassmorphism Container
- Header
- Logo
- General Styling
- Scrollbar
- Fade Animation

Save this as:

static/css/style.css

```
/* =====
AppointmentBot AI
Smart Appointment Assistant
Part 1
===== */

/* Google Font */

@import
url('https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;500;600;700&display=sw
ap');

/* Reset */

*{
  margin:0;
  padding:0;
  box-sizing:border-box;
  font-family:'Poppins',sans-serif;
}

/* Body */

body{

  min-height:100vh;

  display:flex;
```

```

        justify-content:center;

        align-items:center;

        overflow:hidden;

        background:#0f172a;

        position:relative;
    }

    /* =====
    Animated Background
    ===== */

    .background{

        position:fixed;

        width:100%;

        height:100%;

        z-index:-1;

        background:linear-gradient(
            -45deg,
            #1e3a8a,
            #2563eb,
            #4f46e5,
            #7c3aed
        );

        background-size:400% 400%;

        animation:gradientMove 15s ease infinite;
    }

    /* Gradient Animation */

    @keyframes gradientMove{

```

```
0%{

background-position:0% 50%;

}

50%{

background-position:100% 50%;

}

100%{

background-position:0% 50%;

}

}

/* =====
Main Container
===== */

.container{

width:900px;

max-width:95%;

height:92vh;

border-radius:25px;

overflow:hidden;

display:flex;

flex-direction:column;

backdrop-filter:blur(20px);

background:rgba(255,255,255,.15);
```

```

border:1px solid rgba(255,255,255,.2);

box-shadow:

    0 10px 40px rgba(0,0,0,.35);

animation:fadeIn 1s ease;
}

/* =====
Header
===== */

.header{

    display:flex;

    align-items:center;

    gap:20px;

    padding:25px;

    color:white;

    border-bottom:1px solid rgba(255,255,255,.15);

    background:rgba(255,255,255,.08);

}

/* Logo */

.logo{

    width:70px;

    height:70px;

    border-radius:50%;

    background:white;

```

```
display:flex;

justify-content:center;

align-items:center;

font-size:36px;

box-shadow:0 8px 20px rgba(0,0,0,.3);

animation:floatBot 3s infinite ease-in-out;

}

/* Floating Animation */

@keyframes floatBot{

0%{

transform:translateY(0px);

}

50%{

transform:translateY(-10px);

}

100%{

transform:translateY(0px);

}

}

/* Heading */

.header h1{

font-size:30px;
```

```

    font-weight:700;

    letter-spacing:1px;

}

/* Subtitle */

.header p{

    margin-top:5px;

    font-size:15px;

    opacity:.9;

}

/* =====
   Fade Animation
   ===== */

@keyframes fadeIn{

from{

opacity:0;

transform:translateY(20px);

}

to{

opacity:1;

transform:translateY(0px);

}

}

/* =====
   Scrollbar

```

```
===== */
```

```
::-webkit-scrollbar{
```

```
    width:8px;
```

```
}
```

```
::-webkit-scrollbar-track{
```

```
    background:rgba(255,255,255,.08);
```

```
}
```

```
::-webkit-scrollbar-thumb{
```

```
    background:#3b82f6;
```

```
    border-radius:20px;
```

```
}
```

```
::-webkit-scrollbar-thumb:hover{
```

```
    background:#2563eb;
```

```
}
```

```
-----
```

Section	CSS Selector / Code	Purpose	Explanation
1	<code>@import</code>	Import Font	Imports the Poppins Google Font for the entire application.
2	<code>* {}</code>	CSS Reset	Removes default margin and padding, applies <code>box-sizing</code> , and sets the default font.

3	<code>body {}</code>	Body Styling	Centers the chatbot, sets full-screen height, dark background, and hides overflow.
4	<code>.background {}</code>	Animated Background	Creates a colorful animated gradient background behind the chatbot.
5	<code>@keyframes gradientMove</code>	Background Animation	Animates the gradient background continuously.
6	<code>.container {}</code>	Main Container	Creates the chatbot container with glassmorphism effect, rounded corners, and shadow.
7	<code>.header {}</code>	Header	Styles the chatbot header containing the logo and title.
8	<code>.logo {}</code>	Logo	Creates a circular logo with shadow and floating animation.
9	<code>@keyframes floatBot</code>	Floating Animation	Makes the bot logo move up and down continuously.
10	<code>.header h1</code>	Heading	Styles the chatbot title with larger font size and bold weight.
11	<code>.header p</code>	Subtitle	Styles the chatbot subtitle with smaller font and transparency.
12	<code>@keyframes fadeIn</code>	Fade Animation	Provides a smooth fade-in effect when the chatbot loads.
13	<code>::-webkit-scrollbar</code>	Scrollbar	Customizes the scrollbar width and appearance.
14	<code>.chatbox {}</code>	Chat Area	Creates a scrollable message area with vertical layout and spacing.
15	<code>.bot-message, .user-message</code>	Message Layout	Displays bot and user messages using Flexbox.
16	<code>.user-message</code>	User Alignment	Aligns user messages to the right side of the chat window.
17	<code>.avatar {}</code>	Avatar	Styles circular user and bot profile images.

18	<code>.bot-message .avatar</code>	Bot Avatar	Applies blue background and shadow to the bot avatar.
19	<code>.user-message .avatar</code>	User Avatar	Applies green background and shadow to the user avatar.
20	<code>.message {}</code>	Chat Bubble	Styles the message bubble with padding, border radius, and readable text.
21	<code>.bot-message .message</code>	Bot Bubble	Gives bot messages a white background with shadow.
22	<code>.user-message .message</code>	User Bubble	Gives user messages a blue background with white text.
23	<code>.message h3</code>	Welcome Heading	Styles the welcome message heading.
24	<code>.message p</code>	Paragraph	Adds spacing between paragraphs inside chat messages.
25	<code>.message ul</code>	List	Adds indentation to unordered lists.
26	<code>.message li</code>	List Items	Adds spacing between list items.
27	<code>.booking {}</code>	Booking Badge	Displays a green badge for booking predictions.
28	<code>.non-booking {}</code>	Non-Booking Badge	Displays a red badge for non-booking predictions.
29	<code>.typing {}</code>	Typing Indicator	Styles the "Bot is typing..." animation.
30	<code>@keyframes blink</code>	Blink Animation	Makes the typing indicator blink continuously.
31	<code>@keyframes messageAnimation</code>	Message Animation	Animates new chat messages when they appear.
32	<code>.message:hover</code>	Hover Effect	Slightly lifts the chat bubble when the mouse hovers over it.
33	<code>.time {}</code>	Timestamp	Displays the message time with smaller, lighter text.

34	<code>.divider {}</code>	Divider	Styles the separator text between chat sections.
35	<code>.input-area {}</code>	Input Section	Creates the bottom input area for typing messages.
36	<code>.input-area input</code>	Input Box	Styles the text input with rounded corners and padding.
37	<code>.input-area input:focus</code>	Focus Effect	Adds a blue glow when the input field is active.
38	<code>.input-area button</code>	Buttons	Styles all action buttons with circular design.
39	<code>#sendBtn</code>	Send Button	Gives the send button a blue color.
40	<code>#sendBtn:hover</code>	Send Hover	Enlarges the send button slightly when hovered.
41	<code>#clearBtn</code>	Clear Button	Styles the clear button with a red background.
42	<code>#clearBtn:hover</code>	Clear Hover	Enlarges the clear button on hover.
43	<code>.examples {}</code>	Example Buttons	Creates a responsive layout for example prompt buttons.
44	<code>.example-btn</code>	Example Button	Styles example buttons with rounded corners and shadow.
45	<code>.example-btn:ho ver</code>	Example Hover	Changes button color and moves it slightly upward on hover.
46	<code>.footer {}</code>	Footer	Styles the chatbot footer with centered text.
47	<code>.footer p</code>	Footer Text	Adds spacing between footer paragraphs.
48	<code>@media(max-width: 768px)</code>	Tablet Responsive	Adjusts layout, fonts, and button sizes for tablets and smaller screens.
49	<code>@media(max-width: 480px)</code>	Mobile Responsive	Optimizes the chatbot for mobile devices with smaller fonts and logo.

50	<code>@keyframes floatButton</code>	Button Animation	Creates a floating animation for buttons when hovered.
51	<code>.container:hover</code>	Container Hover	Adds a glowing shadow effect when hovering over the chatbot container.
52	<code>::selection</code>	Text Selection	Changes the background and text color when users select text.
53	<code>/* End of CSS */</code>	End Comment	Marks the end of the stylesheet.

=====

Completed in Part 1

- ✓ Google Font
- ✓ Reset CSS
- ✓ Animated Gradient Background
- ✓ Glassmorphism Container
- ✓ Header
- ✓ Logo Animation
- ✓ Fade Animation
- ✓ Custom Scrollbar

Next: Part 2

In Part 2, I'll provide:

- 💬 Chat bubbles
- 🤖 Bot avatar
- 👤 User avatar
- ✨ Typing animation
- 📄 Chat window
- 📩 Message cards
- 😊 Welcome message styling

This will make the chatbot start looking like **ChatGPT/Gemini**.

```
/* =====  
  
AppointmentBot AI  
  
Smart Appointment Assistant  
  
Part 2  
  
Chat Window, Messages, Typing Animation  
  
===== */
```

```
/* =====  
  
Chat Window  
  
===== */
```

```
.chatbox{  
  
    flex:1;  
  
    overflow-y:auto;  
  
    padding:25px;  
  
    display:flex;
```

```
flex-direction:column;
```

```
gap:18px;
```

```
scroll-behavior:smooth;
```

```
}
```

```
/* =====
```

```
Common Message Style
```

```
===== */
```

```
.bot-message,
```

```
.user-message{
```

```
display:flex;
```

```
align-items:flex-start;
```

```
gap:15px;
```

```
animation:messageAnimation .4s ease;
```

```
}
```

```
/* User message aligned right */
```

```
.user-message{
```

```
    justify-content: flex-end;
```

```
}
```

```
/* =====
```

```
Avatar
```

```
===== */
```

```
.avatar{
```

```
    width: 50px;
```

```
    height: 50px;
```

```
    border-radius: 50%;
```

```
display:flex;
```

```
justify-content:center;
```

```
align-items:center;
```

```
font-size:24px;
```

```
color:white;
```

```
flex-shrink:0;
```

```
}
```

```
/* Bot Avatar */
```

```
.bot-message .avatar{
```

```
background:#2563eb;
```

```
box-shadow:0 6px 15px rgba(37,99,235,.35);
```

```
}
```

```
/* User Avatar */
```

```
.user-message .avatar{
```

```
background:#16a34a;
```

```
box-shadow:0 6px 15px rgba(22,163,74,.35);
```

```
}
```

```
/* =====
```

```
Chat Bubble
```

```
===== */
```

```
.message{
```

```
max-width:70%;
```

```
padding:16px 20px;
```

```
border-radius:18px;
```



```
font-size:15px;
```

```
line-height:1.7;
```

```
word-wrap:break-word;
```

```
}
```

```
/* Bot Bubble */
```

```
.bot-message .message{
```

```
background:white;
```

```
color:#111827;
```

```
border-top-left-radius:5px;
```

```
box-shadow:0 10px 20px rgba(0,0,0,.12);
```

```
}
```

```
/* User Bubble */
```

```
.user-message .message{
```

```
background:#2563eb;
```

```
color:white;
```

```
border-top-right-radius:5px;
```

```
box-shadow:0 10px 20px rgba(37,99,235,.30);
```

```
}
```

```
/* =====
```

```
Welcome Message
```

```
===== */
```

```
.message h3{
```

```
margin-bottom:8px;
```

```
color:#2563eb;
```

```
}
```

```
.message p{
```

```
margin-bottom:8px;
```

```
}
```

```
.message ul{
```

```
margin-top:8px;
```

```
padding-left:20px;
```

```
}
```

```
.message li{
```

```
margin-bottom:6px;
```

```
}
```

```
/* =====
```

```
    Booking Badge
```

```
===== */
```

```
.booking{
```

```
    display:inline-block;
```

```
    background:#22c55e;
```

```
    color:white;
```

```
    padding:6px 14px;
```

```
    border-radius:20px;
```

```
    font-size:13px;
```

```
    font-weight:600;
```

```
margin-top:10px;
```

```
}
```

```
/* =====
```

```
Non Booking Badge
```

```
===== */
```

```
.non-booking{
```

```
display:inline-block;
```

```
background:#ef4444;
```

```
color:white;
```

```
padding:6px 14px;
```

```
border-radius:20px;
```

```
font-size:13px;
```

```
font-weight:600;
```

```
margin-top:10px;
```

```
}
```

```
/* =====
```

```
Typing Indicator
```

```
===== */
```

```
.typing{
```

```
display:none;
```

```
color:white;
```

```
padding:12px 25px;
```

```
font-size:15px;
```

```
font-weight:500;
```

```
animation:blink 1.2s infinite;
```

```
}
```

```
/* Blinking */
```

```
@keyframes blink{
```

```
0%{
```

```
opacity:.3;
```

```
}
```

```
50%{
```

```
opacity:1;
```

```
}
```

```
100%{
```

```
opacity:.3;
```

```
}
```

```
}
```

```
/* =====
```

```
    Message Animation
```

```
===== */
```

```
@keyframes messageAnimation{
```

```
0%{
```

```
    opacity:0;
```

```
    transform:translateY(20px);
```

```
}
```

```
100%{
```

```
    opacity:1;
```

```
    transform:translateY(0);
```



```
}
```

```
}
```

```
/* =====
```

```
    Hover Effect
```

```
===== */
```

```
.message:hover{
```

```
    transform:translateY(-2px);
```

```
    transition:.3s;
```

```
}
```

```
/* =====
```

```
    Timestamp
```

```
===== */
```

```
.time{
```

margin-top:8px;

font-size:11px;

opacity:.6;

}

/* =====

Chat Divider

===== */

.divider{

width:100%;

text-align:center;

color:white;

font-size:13px;

margin:12px 0;

```
opacity:.7;
```

```
}
```

```
=====
```

✅ Part 2 Completed

This section adds:

- 💬 Chat window
- 🤖 AI avatar
- 👤 User avatar
- ✨ Chat bubbles
- 🎉 Welcome message styling
- 🟢 Booking badge
- 🔴 Non-Booking badge
- ⌨️ Typing animation
- ✨ Fade-in animation
- 🕒 Timestamp style
- 📄 Smooth scrolling

Next: Part 3 (Final CSS)

Part 3 will include:

- 📄 Input box
- 🚀 Send button
- 🗑️ Clear button
- 🎨 Example buttons
- 📱 Mobile responsive design
- ✨ Hover animations
- 📌 Footer

- 🌙 Dark mode polish
- 🎨 Final professional UI (ChatGPT/Gemini style)

After Part 3, your chatbot UI will be fully styled and ready to connect with your Flask backend.

Perfect! Below is the **final Part 3** of your **style.css**. Add this **after Part 2**.

This section styles:

- Input box
- Send button
- Clear button
- Example buttons
- Footer
- Mobile responsive layout
- Hover animations
- Focus effects

```
/* =====
```

AppointmentBot AI

Smart Appointment Assistant

Part 3

Input Area, Buttons, Footer, Responsive Design

```
===== */
```

```
/* =====
```

Input Area

```
===== */
```

```
.input-area{

    display:flex;

    align-items:center;

    gap:12px;

    padding:20px;

    background:rgba(255,255,255,.08);

    border-top:1px solid rgba(255,255,255,.12);

}
```

```
/* Input Box */
```

```
.input-area input{

    flex:1;
```

height:55px;

border:none;

outline:none;

border-radius:30px;

padding:0 22px;

font-size:16px;

background:rgba(255,255,255,.95);

color:#111827;

transition:.3s;

}

/* Focus Effect */

.input-area input:focus{

```
box-shadow:0 0 15px rgba(37,99,235,.45);
```

```
}
```

```
/* =====
```

```
Buttons
```

```
===== */
```

```
.input-area button{
```

```
width:55px;
```

```
height:55px;
```

```
border:none;
```

```
border-radius:50%;
```

```
cursor:pointer;
```

```
color:white;
```

```
font-size:20px;
```

```
transition:.3s;
```

```
}
```

```
/* Send Button */
```

```
#sendBtn{
```

```
background:#2563eb;
```

```
}
```

```
#sendBtn:hover{
```

```
background:#1d4ed8;
```

```
transform:scale(1.08);
```

```
}
```



```
/* Clear Button */
```

```
#clearBtn{
```

```
    background:#ef4444;
```

```
}
```

```
#clearBtn:hover{
```

```
    background:#dc2626;
```

```
    transform:scale(1.08);
```

```
}
```

```
/* =====
```

```
Example Buttons
```

```
===== */
```

```
.examples{
```

```
display:flex;
```

```
flex-wrap:wrap;
```

```
gap:12px;
```

```
padding:18px;
```

```
justify-content:center;
```

```
}
```

```
.example-btn{
```

```
border:none;
```

```
cursor:pointer;
```

```
padding:10px 18px;
```

```
border-radius:25px;
```

```
background:white;
```

```
color:#2563eb;
```

```
font-size:14px;
```

```
transition:.3s;
```

```
box-shadow:0 4px 10px rgba(0,0,0,.10);
```

```
}
```

```
.example-btn:hover{
```

```
background:#2563eb;
```

```
color:white;
```

```
transform:translateY(-2px);
```

```
}
```

```
/* =====  
  
Footer  
  
===== */
```

```
.footer{  
  
    text-align:center;  
  
    padding:15px;  
  
    color:white;  
  
    font-size:13px;  
  
    border-top:1px solid rgba(255,255,255,.12);  
  
    background:rgba(255,255,255,.05);  
  
}
```

```
.footer p{  
  
    margin:3px 0;
```

```
}
```

```
/* =====
```

```
    Responsive Design
```

```
===== */
```

```
@media(max-width:768px){
```

```
.container{
```

```
    width:100%;
```

```
    height:100vh;
```

```
    border-radius:0;
```

```
}
```

```
.header{
```

```
    padding:18px;
```

```
}
```

```
.header h1{
```

```
    font-size:24px;
```

```
}
```

```
.header p{
```

```
    font-size:13px;
```

```
}
```

```
.logo{
```

```
    width:55px;
```

```
    height:55px;
```

```
    font-size:28px;
```

```
}
```

```
.message{  
  
    max-width:88%;  
  
    font-size:14px;  
  
}
```

```
.avatar{  
  
    width:42px;  
  
    height:42px;  
  
    font-size:20px;  
  
}
```

```
.input-area{  
  
    padding:12px;  
  
}
```

```
.input-area input{
```

```
    height:50px;
```

```
    font-size:14px;
```

```
}
```

```
.input-area button{
```

```
    width:48px;
```

```
    height:48px;
```

```
    font-size:18px;
```

```
}
```

```
.examples{
```

```
    display:none;
```

```
}
```



```
}
```

```
/* =====  
    Small Mobile  
===== */
```

```
@media(max-width:480px){
```

```
.header{
```

```
    gap:12px;
```

```
}
```

```
.header h1{
```

```
    font-size:20px;
```

```
}
```

```
.logo{
```

```
    width:48px;
```

```
height:48px;
```

```
font-size:24px;
```

```
}
```

```
.message{
```

```
font-size:13px;
```

```
}
```

```
}
```

```
/* =====
```

```
Floating Animation
```

```
===== */
```

```
.input-area button:hover{
```

```
animation:floatButton .4s;
```

```
}
```

```
@keyframes floatButton{
```

```
0%{
```

```
transform:translateY(0);
```

```
}
```

```
50%{
```

```
transform:translateY(-5px);
```

```
}
```

```
100%{
```

```
transform:translateY(0);
```

```
}
```

```
}
```

```
/* =====  
  
    Glow Effect  
  
===== */  
  
.container:hover{  
  
    box-shadow:  
  
    0 15px 45px rgba(37,99,235,.25);  
  
}
```

```
/* =====  
  
    Selection Color  
  
===== */  
  
::selection{  
  
    background:#2563eb;  
  
    color:white;
```

```
}
```

```
/* =====  
  
End of CSS  
  
===== */
```

Your CSS is Complete

You now have a modern chatbot interface with:

- ☒ Animated gradient background
- ☒ Glassmorphism container
- ☒ Floating AI logo
- ☒ ChatGPT-style chat bubbles
- ☒ User & bot avatars
- ☒ Typing indicator
- ☒ Animated send & clear buttons
- ☒ Example prompt buttons
- ☒ Responsive design (mobile + desktop)
- ☒ Smooth hover and focus effects
- ☒ Professional footer

Recommended Next Step

The only missing piece is `static/js/script.js`, which will:

- Send messages to your Flask `/predict` endpoint.
- Display user and bot messages as chat bubbles.
- Show and hide the typing animation.
- Handle the **Enter** key.
- Clear the chat.

- Fill the input when an example button is clicked.
- Auto-scroll to the latest message.
- Add timestamps.
- Show **Booking** (●) and **Non-Booking** (●) badges.

That will make the interface fully interactive and complete.

```
=====
```

```
from flask import Flask, render_template, request, jsonify
import joblib
import re
import nltk
```

```
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
```

```
# Download stopwords (only first run)
nltk.download("stopwords")
```

```
# -----
# Initialize Flask App
# -----
```

```
app = Flask(__name__)
```

```
# -----
# Load Model and Vectorizer
# -----
```

```
model = joblib.load("appointment_bot_model.pkl")
```

```
vectorizer = joblib.load("count_vectorizer.pkl")
```

```
# -----
# NLP Preprocessing
# -----
```

```
stemmer = PorterStemmer()
```

```
stop_words = set(stopwords.words("english"))
```

```

def preprocess(text):

    text = text.lower()

    text = re.sub(r"[^a-zA-Z\s]", "", text)

    words = text.split()

    words = [
        stemmer.stem(word)
        for word in words
        if word not in stop_words
    ]

    return " ".join(words)

# -----
# Home Page
# -----

@app.route("/")

def home():

    return render_template("index.html")

# -----
# Prediction API
# -----

@app.route("/predict", methods=["POST"])

def predict():

    data = request.get_json()

    message = data["message"]

    clean = preprocess(message)

    vector = vectorizer.transform([clean])

    prediction = model.predict(vector)[0]

```

```

# Confidence (LinearSVC has no predict_proba)
response = {

    "prediction": prediction

}

return jsonify(response)

# -----
# Run App
# -----

if __name__ == "__main__":

    app.run(

        host="0.0.0.0",

        port=7860,

        debug=True

    )

=====

```

[app.py](#) for AppointmentBot AI

Section	Code	Explanation
Import Flask	<pre>from flask import Flask, render_template, request, jsonify</pre>	Imports Flask modules to create the web application, render HTML pages, receive user requests, and return JSON responses.

Import Joblib	<code>import joblib</code>	Loads the saved Machine Learning model and CountVectorizer from disk.
Import Regex	<code>import re</code>	Used for text cleaning by removing punctuation and special characters.
Import NLTK	<code>import nltk</code>	Imports the Natural Language Toolkit library for text preprocessing.
Import Stopwords	<code>from nltk.corpus import stopwords</code>	Imports English stopwords such as "is", "the", "and", which are removed during preprocessing.
Import Stemmer	<code>from nltk.stem import PorterStemmer</code>	Imports the Porter Stemmer to reduce words to their root form.
Download Stopwords	<code>nltk.download("stopwords")</code>	Downloads the English stopword dataset (only needed on the first run).
Create Flask App	<code>app = Flask(__name__)</code>	Initializes the Flask web application.
Load Model	<code>model = joblib.load("appointment_bot_model.pkl")</code>	Loads the trained Linear SVM model used for prediction.
Load Vectorizer	<code>vectorizer = joblib.load("count_vectorizer.pkl")</code>	Loads the CountVectorizer used to convert text into numerical features.
Create Stemmer	<code>stemmer = PorterStemmer()</code>	Creates a stemming object for reducing words to their root form.
Load Stopwords	<code>stop_words = set(stopwords.words("english"))</code>	Stores English stopwords in a set for faster lookup.

Preprocessing Function	<code>def preprocess(text):</code>	Defines a function to clean and preprocess user input before prediction.
Convert to Lowercase	<code>text = text.lower()</code>	Converts all text to lowercase for consistency.
Remove Symbols	<code>re.sub(r"[^a-zA-Z\s]", "", text)</code>	Removes numbers, punctuation, and special characters.
Split Words	<code>words = text.split()</code>	Splits the sentence into individual words (tokens).
Remove Stopwords & Stem	<code>for word in words ...</code>	Removes unnecessary words and applies stemming to each remaining word.
Return Clean Text	<code>return " ".join(words)</code>	Joins the processed words back into a single cleaned sentence.
Home Route	<code>@app.route("/")</code>	Defines the default route of the application.
Home Function	<code>return render_template("index.html")</code>	Loads and displays the chatbot interface in the browser.
Prediction Route	<code>@app.route("/predict", methods=["POST"])</code>	Creates a REST API endpoint that accepts POST requests for predictions.
Receive JSON	<code>data = request.get_json()</code>	Receives the user message sent from JavaScript in JSON format.
Extract Message	<code>message = data["message"]</code>	Retrieves the message text from the JSON object.

Preprocess Text	<code>clean = preprocess(message)</code>	Cleans the user's message using the preprocessing function.
Vectorize Text	<code>vector = vectorizer.transform([clean])</code>	Converts the cleaned text into numerical features using CountVectorizer.
Predict Class	<code>prediction = model.predict(vector)[0]</code>	Uses the trained Linear SVM model to predict whether the message is Booking or Non-Booking.
Create Response	<code>response = {"prediction": prediction}</code>	Stores the prediction inside a JSON object.
Return JSON	<code>return jsonify(response)</code>	Sends the prediction back to the frontend as a JSON response.
Run Application	<code>if __name__ == "__main__":</code>	Ensures the Flask server starts only when the file is executed directly.
Start Flask Server	<code>app.run(host="0.0.0.0", port=7860, debug=True)</code>	Starts the Flask development server on port 7860 with debug mode enabled.

=====

[script.js](#)

```
/* =====
AppointmentBot AI
Smart Appointment Assistant
script.js
Part 1
===== */
```

```
/* =====
Get HTML Elements
===== */
```

```
const chatbox = document.getElementById("chatbox");
```

```

const messageInput = document.getElementById("message");

const sendBtn = document.getElementById("sendBtn");

const clearBtn = document.getElementById("clearBtn");

const typing = document.getElementById("typing");

const exampleButtons =
document.querySelectorAll(".example-btn");

const notificationSound = new Audio("/static/audio/notification.mp3");

function playSound(){

    notificationSound.currentTime = 0;

    notificationSound.play().catch(error=>{

        console.log("Sound blocked by browser.");

    });

}

/* =====
   Send Button Click
   ===== */

sendBtn.addEventListener("click", sendMessage);

/* =====
   Press Enter to Send
   ===== */

messageInput.addEventListener("keypress", function(event){

    if(event.key === "Enter"){

        event.preventDefault();
    }

```

```

        sendMessage();

    }

});

/* =====
Main Function
===== */

function sendMessage(){

    const message = messageInput.value.trim();

    if(message === "") return;

    // Show user message
    addUserMessage(message);

    // Clear input box
    messageInput.value = "";

    // Show typing animation
    typing.style.display = "block";

    // Scroll
    scrollTop();

    // Flask Prediction
    predictMessage(message);

}

/* =====
User Chat Bubble
===== */

function addUserMessage(message){

    const div = document.createElement("div");

    div.className = "user-message";

```

```

div.innerHTML = `

    <div class="message">

        ${message}

    </div>

    <div class="avatar">

        

    </div>

`;

chatbox.appendChild(div);

scrollBottom();

}

/* =====
Auto Scroll
===== */

function scrollTop(){

    chatbox.scrollTop = chatbox.scrollHeight;

}

/* =====
AppointmentBot AI
Smart Appointment Assistant
script.js
Part 2
===== */

/* =====

```

Flask Prediction API

===== */

```
async function predictMessage(message){

  try{

    const response = await fetch("/predict",{

      method:"POST",

      headers:{

        "Content-Type":"application/json"

      },

      body:JSON.stringify({

        message:message

      })

    });

    const data = await response.json();

    // Hide typing animation

    typing.style.display = "none";

    // Display Bot Message

    addBotMessage(data.prediction);

    playSound();

    // Play notification sound

  }

}
```

```

catch(error){

    typing.style.display="none";

    addErrorMessage();

    console.error(error);

}

}

/* =====
Bot Response
===== */

function addBotMessage(prediction){

    let badge="";

    let reply="";

    if(prediction==="Booking"){

        badge='<span class="booking">🟢 Booking</span>';

        reply="Great! Your message looks like an appointment booking request.";

    }

    else{

        badge='<span class="non-booking">🔴 Non-Booking</span>';

        reply="This message doesn't appear to be an appointment booking request.";

    }

    const div=document.createElement("div");

    div.className="bot-message";

    div.innerHTML=`

```



```

<div class="avatar">

    

</div>

<div class="message">

    <strong>AppointmentBot AI</strong>

    <br><br>

    ${reply}

    <br><br>

    ${badge}

    <div class="time">

        ${getCurrentTime()}

    </div>

</div>

`;

chatbox.appendChild(div);

scrollBottom();

}

/* =====
Error Message
===== */

function addErrorMessage(){

    const div=document.createElement("div");

```

```
div.className="bot-message";
```

```
div.innerHTML=`
```

```
  <div class="avatar">
```



```
  </div>
```

```
  <div class="message">
```

```
    <strong>Server Error</strong>
```

```
    <br><br>
```

```
    Unable to connect to AppointmentBot AI.
```

```
    <br>
```

```
    Please try again.
```

```
    <div class="time">
```

```
      ${getCurrentTime()}
```

```
    </div>
```

```
  </div>
```

```
`;
```

```
chatbox.appendChild(div);
```

```
scrollBottom();
```

```
}
```

```
/* =====  
Current Time  
===== */
```

```
function getCurrentTime(){
```

```

const now=new Date();

return now.toLocaleTimeString([],{

    hour:"2-digit",

    minute:"2-digit"

});

}

/* =====
AppointmentBot AI
Smart Appointment Assistant
script.js
Part 3
===== */

/* =====
Clear Chat
===== */

clearBtn.addEventListener("click", function(){

    chatbox.innerHTML = `

        <div class="bot-message">

            <div class="avatar">

                

            </div>

            <div class="message">

                <h3>Hello 🙌 </h3>

                <p>

```

Welcome back to
AppointmentBot AI

</p>

<p>

Ask me anything about appointment booking.

</p>

Book appointment tomorrow

Reserve a slot

Can I visit on Monday?

</div>

</div>

`;
`;

});

```
/* =====  
Example Buttons  
===== */
```

```
exampleButtons.forEach(function(button){  
  
    button.addEventListener("click", function(){  
  
        messageInput.value = button.innerText;  
  
        messageInput.focus();  
  
    });
```

```
});
```

```
/* =====  
Auto Focus  
===== */
```

```
window.onload = function(){  
  
    messageInput.focus();  
  
}
```

```
/* =====  
Loading Animation  
===== */
```

```
function showTyping(){  
  
    typing.style.display = "block";  
  
}
```

```
function hideTyping(){  
  
    typing.style.display = "none";  
  
}
```

```
/* =====  
Smooth Scroll  
===== */
```

```
function smoothScroll(){  
  
    chatbox.scrollTo({  
  
        top: chatbox.scrollHeight,  
  
        behavior: "smooth"  
  
    });
```

```
}
```

```
/* =====  
Welcome Animation  
===== */
```

```
setTimeout(function(){  
  
    const welcome = document.querySelector(".bot-message");  
  
    if(welcome){  
  
        welcome.style.animation =  
  
        "messageAnimation .8s ease";  
  
    }  
  
},500);
```

```
/* =====  
Enter Key Animation  
===== */
```

```
messageInput.addEventListener("focus",function(){  
  
    messageInput.style.transition=".3s";  
  
});
```

```
/* =====  
Disable Send Button  
===== */
```

```
function disableButton(){  
  
    sendBtn.disabled = true;  
  
    sendBtn.style.opacity = ".5";
```

```
}
```

```
function enableButton(){
```

```
    sendBtn.disabled = false;
```

```
    sendBtn.style.opacity = "1";
```

```
}
```

```
/* =====  
Optional Sound  
===== */
```

```
// Uncomment if notification sound is available
```

```
/*
```

```
const audio = new Audio("/static/audio/notification.mp3");
```

```
audio.play();
```

```
*/
```

```
/* =====  
Console Message  
===== */
```

```
console.log("=====");
```

```
console.log(" AppointmentBot AI Loaded Successfully ");
```

```
console.log("=====");
```

```
/* =====  
Project Information  
===== */
```

```

console.log("Developer : Sugumar R");

console.log("Frontend : HTML + CSS + JavaScript");

console.log("Backend : Flask");

console.log("Model : CountVectorizer + Linear SVM");

console.log("=====");

```

S.No	Section	Code / Function	Purpose	Explanation
1	Get HTML Elements	<code>document.getElementById()</code>	Access HTML Elements	Retrieves HTML elements such as chatbox, input field, buttons, and typing indicator.
2	Example Buttons	<code>querySelectorAll()</code>	Select Buttons	Selects all example prompt buttons.
3	Notification Sound	<code>new Audio()</code>	Load Audio	Loads the notification sound file for bot replies.
4	Play Sound	<code>playSound()</code>	Play Notification	Plays the notification sound whenever the bot responds.

5	Send Button	<code>sendBtn.addEventListener()</code>	Send Message	Calls <code>sendMessage()</code> when the Send button is clicked.
6	Enter Key	<code>keypress</code> Event	Keyboard Shortcut	Sends the message when the user presses the Enter key.
7	Main Function	<code>sendMessage()</code>	Send User Message	Reads the user's input, displays it, clears the input box, shows typing animation, and sends the message to Flask.
8	Empty Check	<code>if(message === "")</code>	Validation	Prevents empty messages from being sent.
9	User Message	<code>addUserMessage()</code>	Display User Chat	Creates and displays the user's chat bubble with avatar.
10	Auto Scroll	<code>scrollBottom()</code>	Scroll Chat	Automatically scrolls the chat window to the latest message.

11	Flask API	<code>predictMessage()</code>	Connect Backend	Sends the user's message to the Flask backend using the Fetch API.
12	Fetch API	<code>fetch("/predict")</code>	HTTP Request	Sends a POST request to the <code>/predict</code> endpoint.
13	JSON Request	<code>JSON.stringify()</code>	Send Data	Converts the user's message into JSON format before sending it.
14	Receive Response	<code>response.json()</code>	Read Prediction	Receives the prediction returned by the Flask backend.
15	Hide Typing	<code>typing.style.display="none"</code>	Stop Animation	Hides the typing animation after receiving the prediction.
16	Bot Message	<code>addBotMessage()</code>	Display Bot Reply	Displays the bot's prediction as a chat message.
17	Notification Sound	<code>playSound()</code>	Audio Feedback	Plays the notification sound after displaying the bot response.

18	Error Handling	<code>catch(error)</code>	Exception Handling	Displays an error message if the Flask server cannot be reached.
19	Bot Reply	<code>addBotMessage(prediction)</code>	Generate Response	Creates a bot reply based on the prediction result.
20	Booking Check	<code>if(prediction=="Booking")</code>	Classification	Displays the booking response and green badge.
21	Non-Booking Check	<code>else</code>	Classification	Displays the non-booking response and red badge.
22	Bot Avatar	<code></code>	Avatar	Displays the bot profile image.
23	Time Stamp	<code>getCurrentTime()</code>	Message Time	Shows the current time below every bot response.
24	Error Message	<code>addErrorMessage()</code>	Server Error	Displays an error message if prediction fails.
25	Current Time	<code>getCurrentTime()</code>	Time Function	Returns the current time in HH:MM format.

26	Clear Chat	<code>clearBtn.addEventListener()</code>	Reset Chat	Clears all previous messages and restores the welcome message.
27	Example Buttons	<code>exampleButtons.forEach()</code>	Quick Input	Copies the selected example text into the input box.
28	Auto Focus	<code>window.onload</code>	Cursor Focus	Automatically places the cursor inside the message input field.
29	Show Typing	<code>showTyping()</code>	Typing Animation	Displays the typing indicator.
30	Hide Typing	<code>hideTyping()</code>	Typing Animation	Hides the typing indicator.
31	Smooth Scroll	<code>smoothScroll()</code>	Scroll Animation	Smoothly scrolls to the latest chat message.
32	Welcome Animation	<code>setTimeout()</code>	Welcome Effect	Animates the initial welcome message when the page loads.
33	Input Focus	<code>messageInput.addEventListener("focus")</code>	Input Effect	Adds a smooth transition when the input field gains focus.

34	Disable Button	<code>disableButton()</code>	Prevent Multiple Clicks	Disables the Send button while processing.
35	Enable Button	<code>enableButton()</code>	Enable Button	Enables the Send button again after processing.
36	Optional Audio	<code>new Audio()</code>	Alternative Sound	Optional code for playing notification sounds.
37	Console Message	<code>console.log()</code>	Debugging	Prints project loading messages in the browser console.
38	Project Information	<code>console.log()</code>	Project Details	Displays developer name, frontend, backend, and ML model information in the browser console.

Step	Function	Description
1	User enters a message	The user types a message in the input box.
2	<code>sendMessage()</code>	Validates the input and displays the user's message.
3	<code>predictMessage()</code>	Sends the message to the Flask backend using the Fetch API.

4	Flask Backend	Preprocesses the text and predicts Booking or Non-Booking .
5	<code>response.json()</code>	Receives the prediction from the backend.
6	<code>addBotMessage()</code>	Displays the bot's prediction in the chat window.
7	<code>playSound()</code>	Plays the notification sound after the bot response.
8	<code>scrollBottom()</code>	Automatically scrolls to the newest message.
9	<code>getCurrentTime()</code>	Displays the current time below the bot message.
10	User continues chatting	The chatbot repeats the process for every new message.

Step	Process
1	User enters a message in the chatbot.
2	JavaScript sends the message to the Flask backend using the <code>/predict</code> API.
3	Flask receives the message in JSON format.
4	The <code>preprocess()</code> function cleans the text (lowercase, remove symbols, remove stopwords, apply stemming).
5	<code>CountVectorizer</code> converts the cleaned text into numerical features.
6	The trained Linear SVM model predicts whether the message is Booking or Non-Booking .
7	Flask returns the prediction as a JSON response.
8	JavaScript receives the prediction and displays it in the chatbot interface.

=====

Index.html

```
<!DOCTYPE html>
<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">

  <title>AppointmentBot AI</title>

  <link rel="stylesheet"
    href="{{ url_for('static', filename='css/style.css') }}">

  <link rel="preconnect"
    href="https://fonts.googleapis.com">

  <link rel="preconnect"
    href="https://fonts.gstatic.com"
    crossorigin>

  <link
href="https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;500;600;700&display=swap"
rel="stylesheet">

  <link rel="stylesheet"
    href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.6.0/css/all.min.css">

</head>

<body>

<div class="background"></div>

<div class="container">

  <!-- Header -->
```

```
<div class="header">

  <div class="logo">

    

  </div>

  <div>

    <h1>AppointmentBot AI</h1>

    <p>Smart Appointment Assistant</p>

  </div>

</div>

<!-- Chat Box -->

<div id="chatbox" class="chatbox">

  <!-- Welcome Message -->

  <div class="bot-message">

    <div class="avatar">

      

    </div>

    <div class="message">

      <h3>Hello 🖐️</h3>

      <p>

        Welcome to <b>AppointmentBot AI</b>.

      </p>

      <p>
```


I can detect whether a message is an appointment booking request.

</p>

<p>

Try:

</p>

Can I book an appointment tomorrow?

I need to see the doctor on Friday.

Reserve a slot for Monday.

</div>

</div>

</div>

<!-- Typing -->

<div id="typing" class="typing">



AppointmentBot AI is typing...

</div>

<!-- Input Area -->

<div class="input-area">

<input

type="text"

id="message"

placeholder="Type your message here..."

autocomplete="off"

>

<button id="sendBtn">

<i class="fas fa-paper-plane"></i>

</button>

<button id="clearBtn">

<i class="fas fa-trash"></i>

</button>

</div>

<!-- Examples -->

<div class="examples">

<button class="example-btn">

Can I book an appointment tomorrow?

</button>

<button class="example-btn">

Thank you

</button>

<button class="example-btn">

Reserve a slot for Monday

</button>

</div>

```

<!-- Footer -->

<div class="footer">

    <p>

        AppointmentBot AI © 2026

    </p>

    <p>

        Built with Flask • Machine Learning • NLP

    </p>

</div>

</div>

<script src="{{ url_for('static', filename='js/script.js') }}"></script>

</body>

</html>

```

S.N	Section	HTML Code / Tag	Purpose	Explanation
1	Document Type	<!DOCTYPE html>	HTML5 Declaration	Specifies that the document uses HTML5.
2	HTML Tag	<html lang="en">	Root Element	Defines the root of the HTML document and sets the language to English.

3	Head Section	<code><head></code>	Metadata	Contains the page title, CSS files, fonts, and external libraries.
4	Character Encoding	<code><meta charset="UTF-8"></code>	Unicode Support	Supports all international characters and symbols.
5	Responsive Design	<code><meta name="viewport"...></code>	Mobile Support	Makes the webpage responsive on different screen sizes.
6	Page Title	<code><title></code>	Browser Title	Displays AppointmentBot AI on the browser tab.
7	CSS File	<code>style.css</code>	Styling	Loads the external CSS file for the chatbot interface.
8	Google Fonts	<code><link rel="preconnect"></code>	Faster Loading	Improves the loading speed of Google Fonts.
9	Poppins Font	<code><link href="https://fonts.googleapis.com/...></code>	Font Style	Imports the Poppins font family for the application.
10	Font Awesome	<code><link rel="stylesheet"...></code>	Icons	Loads Font Awesome icons such as Send and Delete buttons.
11	Body	<code><body></code>	Main Content	Contains all visible content of the chatbot application.
12	Background	<code><div class="background"></code>	Animated Background	Displays the animated gradient background.

13	Main Container	<code><div class="container"></code>	Chatbot Layout	Holds all chatbot components such as header, chat area, input, and footer.
14	Header	<code><div class="header"></code>	Title Section	Displays the chatbot logo, name, and subtitle.
15	Logo	<code><div class="logo"></div></code>	Bot Icon	Displays the chatbot logo at the top.
16	Heading	<code><h1></code>	Chatbot Name	Displays AppointmentBot AI as the main heading.
17	Subtitle	<code><p></code>	Description	Shows the subtitle Smart Appointment Assistant .
18	Chat Box	<code><div id="chatbox"></code>	Conversation Area	Displays all user and bot messages.
19	Welcome Message	<code><div class="bot-message"></code>	Initial Greeting	Shows the welcome message when the application starts.
20	Bot Avatar	<code><div class="avatar"></div></code>	Bot Profile	Displays the bot icon beside the welcome message.
21	Message Box	<code><div class="message"></code>	Bot Message	Displays the chatbot's welcome text and instructions.
22	Heading	<code><h3>Hello </h3></code>	Greeting	Welcomes the user to the chatbot.
23	Description	<code><p></code>	Information	Explains what the chatbot does.

24	Example List	<code></code>	Sample Inputs	Shows example appointment booking messages.
25	Typing Indicator	<code><div id="typing"></code>	Loading Status	Displays AppointmentBot AI is typing... while waiting for the backend response.
26	Input Area	<code><div class="input-area"></code>	User Input	Contains the input box and action buttons.
27	Text Box	<code><input></code>	User Message	Allows users to type appointment-related messages.
28	Send Button	<code><button id="sendBtn"></code>	Send Message	Sends the user's message to the Flask backend.
29	Send Icon	<code><i class="fas fa-paper-plane"></code>	Icon	Displays the paper-plane icon inside the Send button.
30	Clear Button	<code><button id="clearBtn"></code>	Clear Chat	Clears all messages from the chat window.
31	Trash Icon	<code><i class="fas fa-trash"></code>	Icon	Displays the trash icon inside the Clear button.
32	Example Buttons	<code><div class="examples"></code>	Quick Inputs	Displays predefined sample messages for easy testing.
33	Example Button	<code><button class="example-btn"></code>	Sample Message	Inserts a sample appointment message into the input field.

34	Footer	<code><div class="footer"></code>	Footer Section	Displays copyright and technology information.
35	Footer Text	<code><p></code>	Copyright	Shows AppointmentBot AI © 2026 .
36	Technology Text	<code><p></code>	Technologies Used	Displays Built with Flask • Machine Learning • NLP .
37	JavaScript File	<code><script src="{{ url_for('static', filename='js/script.js') }}"></script></code>	Load JavaScript	Loads the external <code>script.js</code> file, which handles chatbot interactions and communication with the Flask backend.

Step	Component	Description
1	Load HTML	The browser loads the <code>index.html</code> page.
2	Load CSS	<code>style.css</code> is loaded to style the chatbot interface.
3	Load Fonts & Icons	Google Fonts and Font Awesome icons are loaded.
4	Display Header	The chatbot logo, title, and subtitle are shown.
5	Display Welcome Message	The bot greets the user with sample appointment messages.
6	User Input	The user types a message into the input box.
7	Click Send	The Send button submits the message to the JavaScript code.

- | | | |
|-----------|----------------------|---|
| 8 | JavaScript Execution | <code>script.js</code> sends the message to the Flask backend for prediction. |
| 9 | Display Response | The chatbot displays the prediction received from the backend. |
| 10 | Footer | The application footer displays copyright and technology information. |
